Managed Services Change Request Tracker

A role-based workflow tool for managed service providers handling website/content updates

Summary:

- Website update requests were happening in fragmented ways: email, chat, loose notes
- That made it hard to track status, preserve context, separate internal vs client-visible communication, and manage attachments
- I built a structured request tracker that turns ad hoc change requests into trackable tickets

Built with Next.js, Supabase, Tailwind
Live demo: https://cms-change-request-tracker.aadityacgupta.workers.dev/login
You can find the logins for the demo on the last page

The Workflow Problem I Noticed

Operational pain

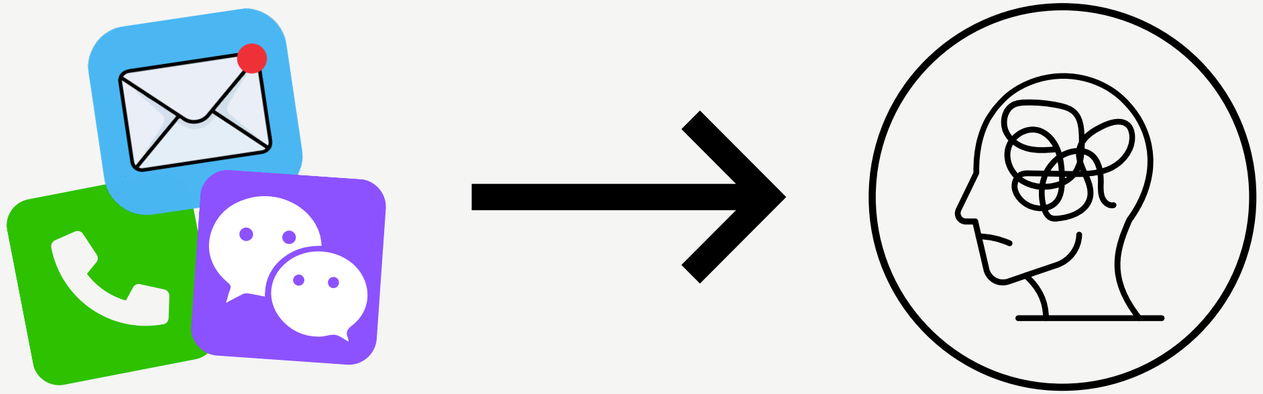
- Requests came in through multiple channels
- No single source of truth
- Easy to lose context or attachments

Internal team pain

- No separation between client-facing updates and internal notes
- Hard to triage across multiple websites/clients
- Difficult to prioritize and audit requests

Client experience pain

- Unclear status
- Follow ups happened manually
- Limited visibility into progress



Product Principles that Shaped the Solution

Structure over ambiguity
Standardized tickets instead of freeform scattered requests

Role-aware visibility
Clients and internal users should not see the same things

Context at the point of action
Users should understand the page they are changing without leaving the tool

Trackability over memory
The system should preserve status, comments, files, and ownership over time

How the Product Works

A Simple Flow:

Client submits request → Internal team triages → Status updates → Comments / collaboration → Resolution / follow up

The Two Roles:

Client Portal

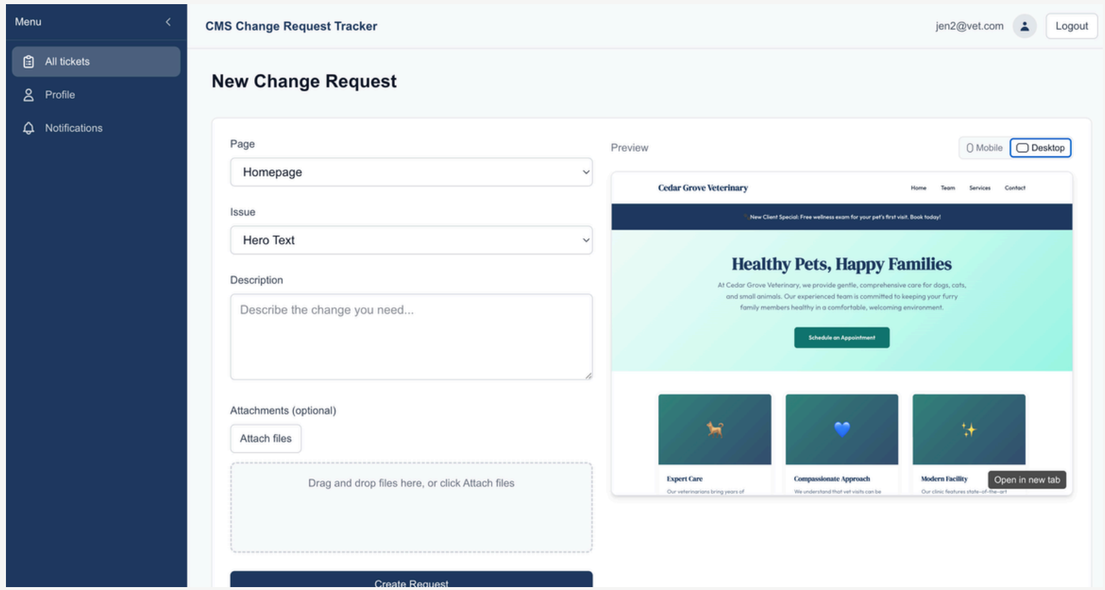
- Create Tickets
- View Status
- Add Comments
- Upload Attachments
- Request Follow Ups

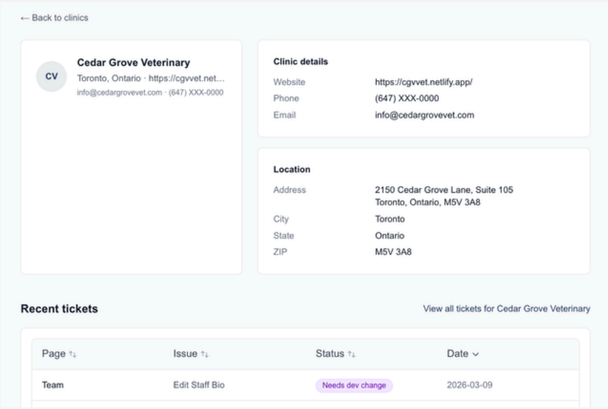
Internal Portal

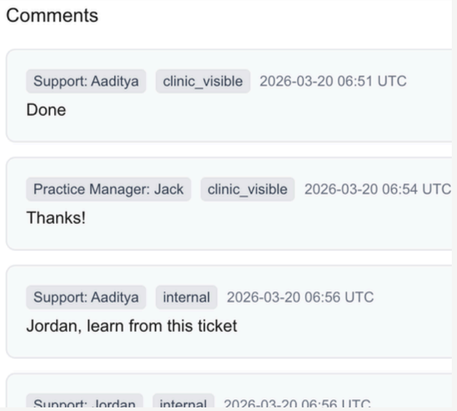
- See All Clients
- Manage Ticket Pipeline
- Change Statuses
- Add Internal-Only Comments
- View Ticket Analytics

Features I Prioritized and the Logic behind them

Feature	Why It Matters
Embedded page preview with iframe	Reduces context switching and helps users request the right change on the right page
Mobile / desktop preview toggle	Reflects how websites are actually experienced across devices; avoids assuming desktop-only workflows
Comments on tickets	Keeps clarifications inside the workflow instead of moving back to email/chat
Internal-only comments	Preserves space for operational notes without exposing everything to clients
Clinic detail view + associated tickets	Gives internal users full context when managing a specific client
Client follow-up on resolved tickets	Makes resolution reversible when the real-world outcome doesn't match expectations







Technical Design and Implementation Choices

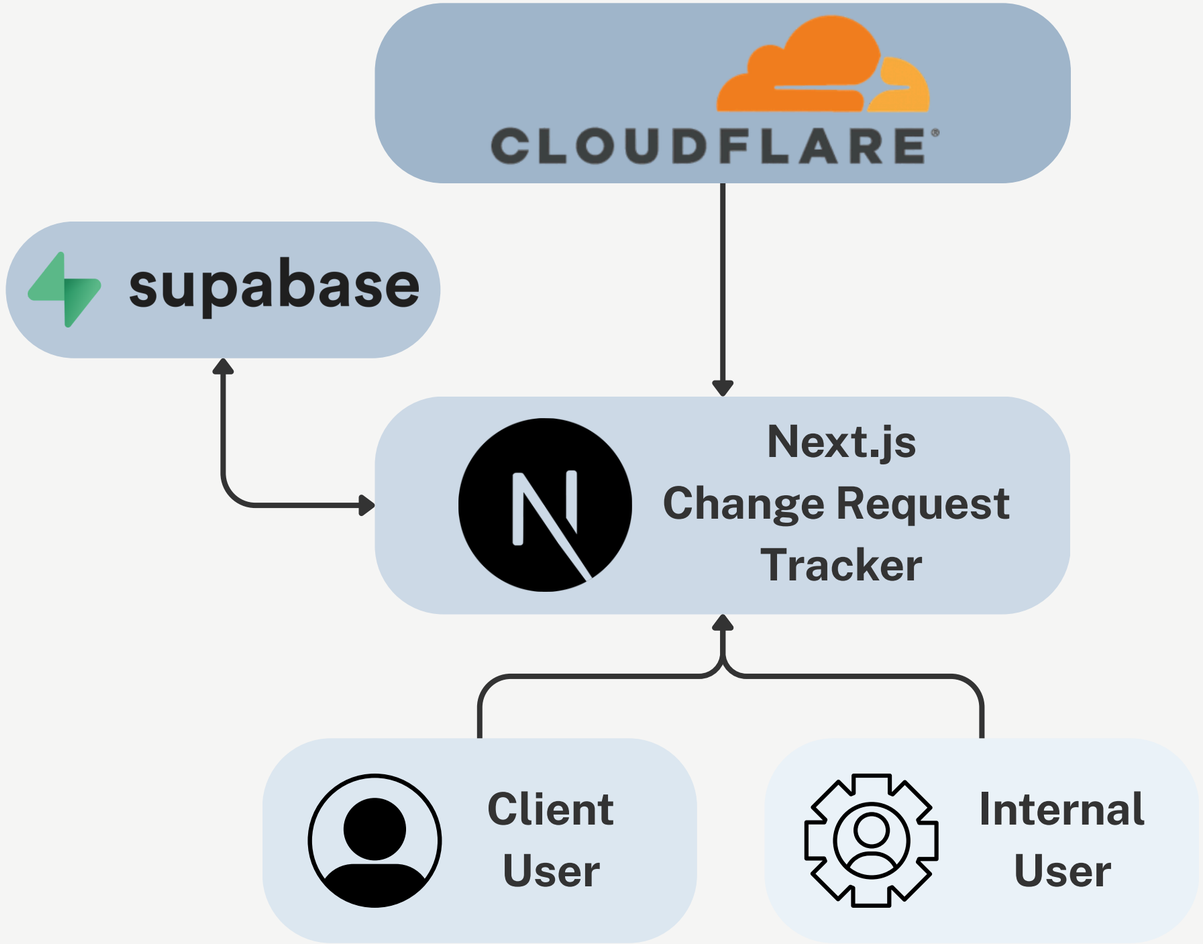
I chose tools that let me focus on workflow design, not infrastructure overhead.

Tech stack

- **Next.js** for app structure and interactive UI
- **Supabase** for auth, Postgres, storage, and row-level security
- **Tailwind** CSS for fast UI iteration
- **Cloudflare** for deployment
- **GitHub** for version control
- **Netlify** for hosting demo websites used in preview

Why this stack

- Fast iteration for MVP and beyond
- Auth + data + storage in one backend platform
- Role-based access through RLS
- Easy deployment and demoability



What I Learned by Shipping this Product

Previewing live sites required practical tradeoffs because responsive previews and iframe behavior are not always straightforward

I learned to separate “good product idea” from “technically reliable implementation”

Role-based visibility required careful thinking around data access, not just UI states

Deployment forced me to understand environment variables, build/runtime differences, and production debugging

Projected Operational Impact

For a program spanning 10 phases and approximately 1,500 total change requests, this tool is estimated to save between 150 and 250 hours of operational effort.

Scenario	Time Saved Per Ticket	Total Hours Saved
Conservative	6 min	150 hrs
Mid	8 min	200 hrs
Upper	10 min	250 hrs

The savings come from four specific workflow changes:

- Structured intake reduces back-and-forth clarification
- Comments and attachments stay attached to the request — nothing gets lost
- Internal vs. client-visible notes are already separated, removing manual filtering
- Status is always visible, eliminating manual chasing

Estimate is modeled based on a real program's ticket volume. Time savings per ticket reflect conservative assumptions about reduced coordination overhead.

Current Outcome and Next Steps

Current outcome

- Working live demo
- Role-based portals
- End-to-end request workflow
- Fictional clinics and demo websites to showcase the product safely

What's Next

- Mobile Friendly UI
- AI asst. for drafting ticket description
- Stronger Analytics
- Workflow tool integrations (Slack, Jira, etc.)
- SLA tracking and audit history

Demo Accounts

Emails	Passwords	User
admin@crt.com	admin1234	Internal
emp@crt.com	emp1234	Internal
jack1@vet.com	jack1	Client
jen2@vet.com	jen2	Client
jojo3@vet.com	jojo3	Client